

Oracle Forms Modernization Proposal

Scope: migrate customer search, customer details, invoices and payments from Oracle Forms to a modern browser-based solution while preserving proven Oracle business logic.

A. Proposed architecture

Browser	API / application	Data
Angular or Vue	Java REST service	Oracle Database
Responsive components	Stateless, container-ready	PL/SQL packages and SQL

HTTPS connects the browser to the backend. JDBC and controlled PL/SQL package APIs connect Java to Oracle. Authentication integrates with the organization identity provider (for example Entra ID through OpenID Connect). Authorization is enforced at API level and, where appropriate, inside database package APIs. Several stateless Java instances can run behind a load balancer.

B. Frontend structure

The frontend starts with a customer search page and a customer-details workspace. The workspace contains separate tabs or sections for general details, invoices and payments. Shared components provide navigation, loading and error states, paged tables and access-denied handling. Search input is validated in the browser, but the backend remains authoritative. Large result sets use server-side paging and filtering.

C. Backend integration approach

Expose use-case-oriented REST endpoints such as GET /customers, GET /customers/{id}, GET /customers/{id}/invoices and GET /customers/{id}/payments. Controllers map HTTP, services coordinate use cases and authorization, and repositories/adapters call Oracle views and PL/SQL packages through JDBC. The browser never connects directly to Oracle and never receives database credentials.

One business operation should normally map to one backend request and one controlled database transaction. Avoid chatty row-by-row JDBC calls; keep data-intensive processing set-based in SQL and PL/SQL.

D. Migration approach

Use incremental migration (strangler pattern), not a big-bang rewrite. First inventory Forms modules, triggers, reports, roles and database dependencies. Create characterization tests for critical current behavior. Introduce Java APIs around stable PL/SQL packages and migrate one vertical flow at a time: customer search, details, invoices and payments. During coexistence, route selected functions/users to the new UI and retain Forms as fallback. Compare outputs, audit records and performance before retiring each module.

Oracle Forms Modernization Proposal - responsibilities and risk

E. Functionality remaining in Oracle / PL/SQL

Retain mature, data-intensive and transaction-critical logic close to the data: complex SQL, bulk processing, consistency rules, constraints, reference data, calculation packages and operations that must be atomic across several tables. Existing tested PL/SQL should be reused unless there is a clear business or technical reason to rewrite it.

F. Functionality moved outside the database

Move presentation, navigation and client interaction to Angular/Vue. Put API contracts, identity-provider integration, coarse-grained authorization, workflow orchestration, external-service integration, HTTP error mapping, observability and application-level caching in Java. Duplicate only lightweight UI validation in the frontend; do not move authoritative validation exclusively to the browser.

G. Main risks and challenges

Risk	Mitigation
Undocumented logic in Forms triggers/reports	Inventory modules and create behavior/regression tests before migration.
Behavior or transaction differences	Preserve package APIs, define transaction boundaries and compare outputs/audit data.
Chatty API and database load	Use coarse-grained endpoints, paging, set-based SQL and performance tests.
Security and role-mapping regressions	Threat modelling, Entra ID/OIDC integration, least privilege and security review.
Long coexistence of two UIs	Migrate by business flow, define owners and an explicit decommissioning plan.
User adoption	Pilot rollout, training, feedback and measured usability improvements.

Recommendation: preserve valuable Oracle logic, introduce a reusable Java API layer, and replace Forms incrementally with a browser UI. This balances migration speed, operational safety and long-term architectural flexibility.

Prepared as part of the Oracle / Java / TypeScript technical assignment.